



Università di Genova
Artificial Intelligence for Robotics II

Planetary Rover Memory Constraints and Data Management

Classical PDDL and PDDL+ Planning Report

Student: Endri Gjinaj
Student ID: 5418701
Academic year: 2025-2026

June 18, 2026

Abstract

This report presents a planning model for the assignment *D1-V8: Planetary Rover - Memory Constraints and Data Management*. A single rover navigates through a known map, collects scientific datasets, stores them in limited onboard memory, and returns to a base station to offload them. The first part uses classical PDDL to model movement, data collection, explicit numeric memory usage and offloading. The second part extends the model with PDDL+ processes and events, representing continuous data encoding, continuous memory corruption, automatic encoding completion and automatic data loss. The experiments include both solvable and unsolvable cases. The classical instances demonstrate that memory capacity changes collection order and offloading strategy. The PDDL+ instances demonstrate that timing can make a mission infeasible even if the map is connected and memory capacity is sufficient.

1 Introduction and Background

Autonomous planetary rovers must plan under constraints. A rover may be able to reach several scientific sites, but it cannot necessarily store all the observations at once. Onboard memory is limited, data items may have different sizes, and offloading can only be performed at a base station. The planning problem is therefore not only to find a path, but also to decide which data to carry together and when to return to free memory.

PDDL separates a planning task into a *domain* and a *problem*. The domain defines reusable concepts: types, predicates, numeric functions and actions. The problem defines a particular mission: objects, initial facts, numeric values and goals. Numeric fluents are necessary in this project because memory is a quantity. A Boolean predicate such as `memory-full` would not distinguish between 4, 6, 8 or 10 units of used memory.

Classical PDDL only changes the world through actions. This is not enough to represent autonomous time-dependent phenomena such as data corruption. PDDL+ extends PDDL with *processes*, which continuously change numeric values while active, and *events*, which fire automatically when their conditions become true. The project uses ENHSP to test both the numeric classical model and the PDDL+ extension.

1.1 Project Objectives and Contributions

The project is designed to satisfy both parts of the assignment. The main contributions are summarized in Table 1.

Table 1: Project objectives and how they are addressed in the implementation.

Assignment objective	Implementation choice
Represent limited rover memory	Memory is modelled with numeric fluents: <code>used-memory</code> , <code>memory-capacity</code> , and <code>data-size</code> .
Ensure actions respect memory capacity	The <code>collect-data</code> action contains a numeric precondition enforcing that used memory plus new data size does not exceed capacity.
Test a sufficient-memory case	Classical Problem 1 validates the basic move–collect–offload sequence.
Test a constrained-memory case	Classical Problem 2 uses datasets of sizes 4, 4, 10, and 6 with capacity 10, forcing solo transport, exact filling, and partial offloading.
Introduce memory degradation over time	The PDDL+ model uses a continuous corruption process active while data is stored.
Trigger automatic data loss	The PDDL+ model includes a <code>data-loss</code> event when corruption reaches the dataset-specific limit.
Discuss abstraction and limitations	The discussion explains symbolic memory modelling, omitted storage details, deterministic corruption, instantaneous movement, and possible extensions.

2 Classical PDDL Model

The classical domain contains three types: **rover**, **location** and **data**. The main predicates are **at**, **connected**, **base**, **data-at**, **collected**, **stored** and **offloaded**. Memory is represented by three numeric functions:

```
(:functions
  (used-memory ?r - rover)
  (memory-capacity ?r - rover)
  (data-size ?d - data)
)
```

The key capacity condition in the **collect-data** action is:

$$\text{used-memory}(r) + \text{data-size}(d) \leq \text{memory-capacity}(r). \quad (1)$$

In PDDL syntax, this is written as:

```
(>= (memory-capacity ?r)
     (+ (used-memory ?r) (data-size ?d)))
```

Table 2 summarizes the three classical actions.

Table 2: Classical PDDL actions.

Action	Main preconditions	Effects
move	Rover is at the source and the target is connected.	Updates the rover location.
collect-data	Rover is at the data site, data is not collected, and Eq. 1 holds.	Marks data as collected and stored; increases used memory.
offload-data	Rover is at a base and the data is stored.	Marks data as offloaded; removes stored state; decreases used memory.

3 Classical Problem Instances

The first classical problem is a minimal validation instance. It has one base, one site and one dataset of size 4. The rover capacity is 10, so the data can be collected safely. The expected action chain is: move from the base to the site, collect the data, return to the base and offload it.

The second classical problem is the constrained mission. The map is linear and the rover capacity is 10. The data sizes are designed so that memory management matters: **data3** has size 10 and must travel alone; **data4** has size 6 and can be combined with one size-4 dataset; two size-4 datasets can also travel together.

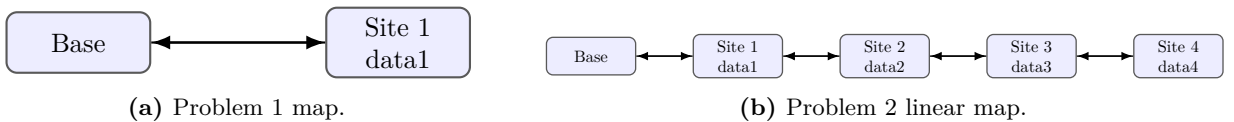


Figure 1: Maps reconstructed from the PDDL initial-state **connected** and **data-at** facts.

The linear topology in Problem 2 was chosen deliberately. The focus of this project is not to create a complex navigation benchmark, but to isolate the effect of onboard memory constraints. A fully connected map would allow the rover to return directly from every site to the base, reducing the importance of planning when to offload data. The linear map prevents direct shortcuts while remaining easy to interpret. Therefore, the route still matters, but the main difficulty remains the

memory-management decision: which datasets can be stored together and when the rover must return to the base.

Table 3: Dataset sizes in classical Problem 2.

Dataset	data1	data2	data3	data4
Location	site1	site2	site3	site4
Size	4	4	10	6

The ENHSP plan for Problem 2 uses the memory capacity actively. It collects and offloads **data3** alone, later combines **data4** with **data2**, then partially offloads and reuses freed memory to collect **data1**. Figure 2 is computed from that plan and Table 3.

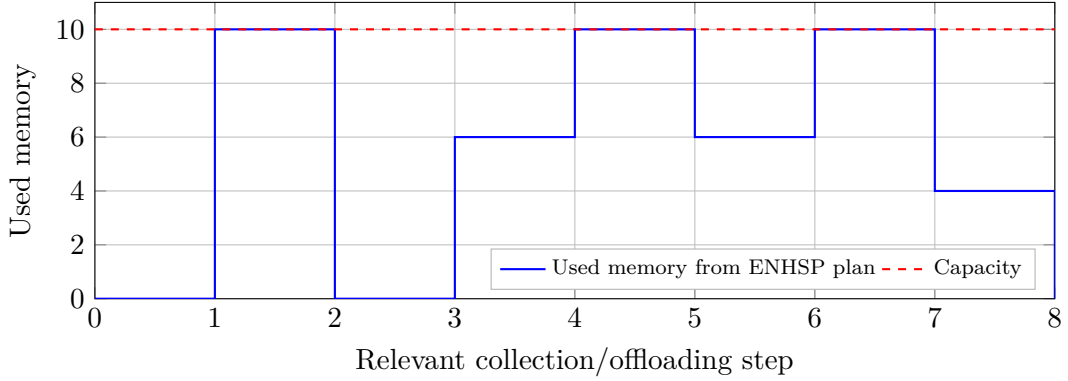


Figure 2: Real memory evolution reconstructed from the classical Problem 2 solver plan. The sequence is: collect **data3**; offload **data3**; collect **data4**; collect **data2**; offload **data2**; collect **data1**; offload **data4**; offload **data1**.

This plot is important because it proves that memory is not a superficial predicate. The planner reasons with numeric occupancy, exact capacity and reuse of freed memory.

3.1 Walkthrough of the Classical Plan Output

The output for Classical Problem 1 behaves as expected. The rover first moves from the base to the only scientific site, collects **data1**, returns to the base, and offloads the data. This confirms the basic action chain:

`move → collect-data → move → offload-data.`

The output for Classical Problem 2 is more interesting because the planner must respect the memory capacity. The generated plan first sends the rover to collect **data3**. Since **data3** has size 10 and the rover capacity is also 10, this dataset occupies the entire memory and must be returned to the base alone. After offloading it, the rover can reuse the full memory capacity.

In the next part of the plan, the rover collects **data4** of size 6 and then **data2** of size 4. This exactly fills the memory:

$$6 + 4 = 10.$$

The rover then returns to the base and offloads only **data2**, leaving **data4** stored. This partial offloading is important because it shows that the planner is not simply emptying all memory every time it reaches the base. Instead, it reuses the freed 4 units of memory to collect **data1**. At the end, both remaining datasets are offloaded. This behaviour matches the intended memory-management trade-off.

4 PDDL+ Extension

The PDDL+ version keeps the classical movement and memory constraints but extends the lifecycle of data. Stored data must become **encoded** before offloading. While it is stored, two continuous processes may be active: encoding progress increases and corruption increases. If corruption reaches the limit first, an automatic **data-loss** event marks the dataset as lost.

The use of PDDL+ is motivated by the fact that corruption and encoding are not controlled directly by the rover as instantaneous actions. Once a dataset is stored, corruption should increase automatically with time. Similarly, encoding progress should evolve while the data remains stored. These behaviours are therefore represented as processes rather than actions. Events are used for threshold-based state changes: when encoding progress reaches the required value, the data automatically becomes encoded; when corruption reaches the limit, the data is automatically lost. Numeric fluents are used because both encoding and corruption are continuous quantities rather than Boolean facts.

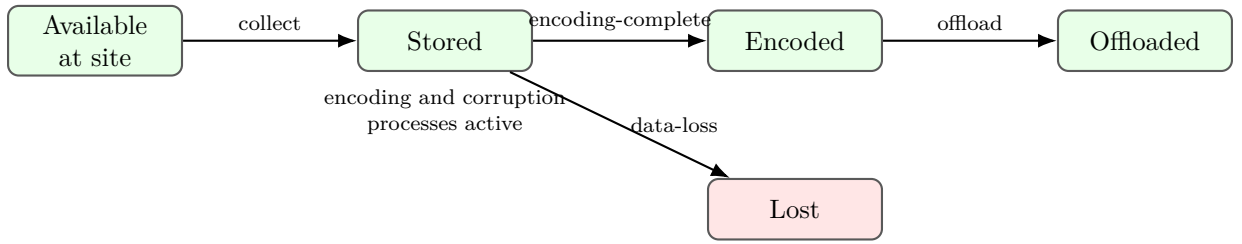


Figure 3: PDDL+ data lifecycle derived from the implemented predicates, process preconditions and event effects.

The relevant numeric functions are:

```

(corruption ?d)
(corruption-limit ?d)
(corruption-rate ?d)
(encoding-progress ?d)
(encoding-required ?d)
(encoding-rate ?d)

```

The corruption process is:

```

(increase (corruption ?d)
  (* #t (corruption-rate ?d)))

```

The encoding process is:

```

(increase (encoding-progress ?d)
  (* #t (encoding-rate ?d)))

```

For each dataset, the idealized encoding and loss times are:

$$T_d^{enc} = \frac{E_d^{required} - E_d(0)}{r_d^{enc}}, \quad T_d^{loss} = \frac{C_d^{limit} - C_d(0)}{r_d^{corr}}. \quad (2)$$

These expressions are obtained directly from the linear PDDL+ continuous effects: since encoding and corruption increase at constant dataset-specific rates, the time required to reach a threshold is the remaining amount divided by the corresponding rate. The **stored-by** predicate was added to associate each stored dataset with the rover carrying it. This is necessary because the **data-loss** event must free the correct rover memory when a dataset becomes lost.

5 PDDL+ Problem Instances

Table 4 gives the numeric values used in the PDDL+ instances. The timing interpretation is calculated directly from Equation 2.

Table 4: PDDL+ dataset parameters and timing interpretation.

Problem	Data	Size	C_{lim}	r_c	E_{req}	r_e	Interpretation
Plus 1	d1	4	10	1.0	2	1.0	safe: $T^{enc} = 2 < T^{loss} = 10$
Plus 2	d1	4	2	1.0	4	1.0	impossible: loss at $t = 2$ before encoding at $t = 4$
Plus 3	d1	4	8	0.5	4	2.0	safe
Plus 3	d2	6	6	1.0	3	1.0	safe
Plus 3	d3	5	2	1.5	1	1.0	urgent but feasible
Plus 4	d1	4	8	0.5	4	2.0	feasible
Plus 4	d2	6	6	1.0	3	1.0	feasible
Plus 4	d3	5	2	2.0	4	1.0	impossible: loss at $t = 1$
Plus 4	d4	5	3	1.0	5	0.5	impossible: loss at $t = 3$

Figure 4 shows the two single-dataset timing cases. These are real plots computed from the PDDL+ problem files: Plus 1 has $E_{req} = 2$, $C_{lim} = 10$, $r_e = 1$, $r_c = 1$; Plus 2 has $E_{req} = 4$, $C_{lim} = 2$, $r_e = 1$, $r_c = 1$.

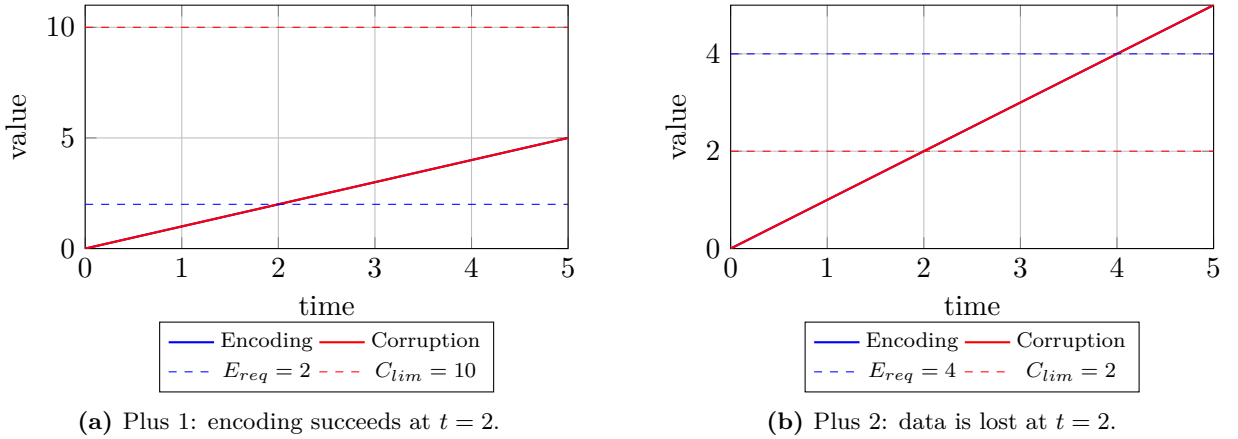


Figure 4: Timing plots computed from PDDL+ numeric initial values. In the failure case, the automatic data-loss event is triggered before encoding can complete.

5.1 Walkthrough of the PDDL+ Plan Output

The PDDL+ outputs show the effect of time-dependent processes. In Plus 1, the rover collects the dataset and then the plan contains a waiting interval. During this waiting time, the **data-encoding** process increases the encoding progress and the **memory-corruption** process increases the corruption value. Encoding reaches its required value at time 2, while corruption would only reach its limit at time 10. Therefore, the automatic **encoding-complete** event can occur before **data-loss**, and the rover can offload the data successfully.

In Plus 2, the same structure becomes impossible. The data requires 4 time units to become encoded, but corruption reaches its limit after only 2 time units. The **data-loss** event therefore fires before the dataset can become offloadable. Since the goal requires the data to be offloaded and not lost, ENHSP correctly reports the problem as unsolvable.

Plus 3 verifies that the PDDL+ model works with multiple datasets. Several stored datasets can have active encoding and corruption processes at the same time, each with its own rate and threshold.

The solved output shows that the planner can still find an ordering that offloads all required data safely.

Plus 4 is intentionally unsolvable. Although some datasets are feasible, **data3** and **data4** cannot be encoded before their corruption limits are reached. Since the goal requires all datasets, the complete mission cannot be solved. This output behaves as expected and confirms that the data-loss event has a real impact on planning.

6 Results

The outputs are stored in the repository under **plans/**. Clean plan summaries are stored next to raw ENHSP outputs. Table 5 summarizes the six instances.

The raw ENHSP outputs were kept in the repository to make the experiments reproducible. For solved instances, the output contains a valid sequence of actions under **Found Plan**. For unsolvable instances, the output reports that no plan can satisfy the goal under the given memory and timing constraints. The unsolvable outputs are part of the experimental results, not execution errors, because they correspond to deliberately infeasible PDDL+ timing configurations.

Table 5: Summary of results and main modelling lesson.

Instance	Model	Result	Main lesson
Problem 1	PDDL	Solved	Basic move, collect, return and offload chain.
Problem 2	PDDL	Solved	Numeric memory forces batching, solo transport and partial offloading.
Plus 1	PDDL+	Solved	Encoding finishes before corruption limit.
Plus 2	PDDL+	Unsolvable	Data-loss event fires before encoding completion.
Plus 3	PDDL+	Solved	Multiple stored datasets evolve concurrently with different rates.
Plus 4	PDDL+	Unsolvable	Some required datasets are temporally impossible to save.

Table 6: Validation of the main required planning features.

Feature	Evidence in the model and experiments
Numeric memory capacity	Memory is represented using <code>used-memory</code> , <code>memory-capacity</code> , and <code>data-size</code> .
Capacity-aware collection	<code>collect-data</code> checks that used memory plus new data size does not exceed capacity.
Sufficient-memory case	Classical Problem 1 is solved with a simple move-collect-return-offload plan.
Memory-constrained case	Classical Problem 2 forces solo transport, exact memory filling, and partial offloading.
Memory reuse	After offloading data, the planner reuses freed memory to collect additional datasets.
Continuous degradation	The PDDL+ corruption process increases corruption while data is stored.
Automatic data loss	The <code>data-loss</code> event fires when corruption reaches the dataset-specific limit.
Timing-dependent feasibility	Plus 2 and Plus 4 are unsolvable because some data is lost before it can be encoded.

The unsolvable results are not failures of the implementation. In Plus 2, no action sequence can offload the data safely because the data is lost at time 2 and only becomes encoded at time 4. In Plus 4, `data3` and `data4` have the same kind of timing conflict. Since the goal requires all datasets to be offloaded and not lost, the mission is impossible under the model.

7 Discussion

This section discusses the abstraction of information storage and the limitations of modelling rover memory symbolically. The model is not intended to simulate a real file system or a real rover computer. Instead, it captures the aspects of memory management that are relevant for planning: how much memory is available, which datasets are stored, whether data has been encoded, and whether it has been safely offloaded or lost.

7.1 Abstraction of Information Storage

The model abstracts real storage into task-level symbols and numeric resources. A scientific observation is represented as an object of type `data`; its memory requirement is represented by the numeric fluent `data-size`; and the rover memory state is represented by `used-memory` and `memory-capacity`. The state of each dataset is represented using predicates such as `collected`, `stored`, `encoded`, `offloaded` and `lost`.

This abstraction is appropriate because the planning question is not which exact memory address stores each byte, but whether the rover can carry the required data and deliver it to the base. For this reason, the model ignores low-level implementation details and keeps only the information needed for task planning. In the classical model, this means checking whether a dataset fits in the remaining memory. In the PDDL+ model, this also means checking whether the dataset can be encoded before corruption reaches its limit.

Numeric fluents are essential for this abstraction. A purely symbolic model could use predicates such as `memory-empty` or `memory-full`, but this would not distinguish between partial memory states. In Problem 2, the planner must reason about memory values 0, 4, 6 and 10. This allows it to combine datasets, exactly fill the memory, offload only part of the stored data, and reuse freed memory later in the plan.

The PDDL+ extension adds another layer to the abstraction. Corruption and encoding are represented as continuous numeric quantities instead of Boolean facts. This makes it possible to model

the timing conflict between data processing and data degradation. A dataset is not simply “good” or “bad” from the beginning; it becomes lost only when its corruption reaches a threshold, and it becomes offloadable only when encoding reaches the required value.

7.2 Limitations and Their Consequences

The model does not include real file-system details such as folders, memory addresses, fragmentation, compression, allocation blocks or deletion overhead. Dataset sizes are fixed and known in advance, while real observations may vary depending on sensor settings, resolution, compression and environmental conditions. As a consequence, the planner reasons with idealized data sizes rather than uncertain or dynamically changing storage requirements.

Offloading is modelled as an instantaneous action. This means that the model does not represent communication bandwidth, contact windows, packet loss, signal quality or transfer duration. In a more realistic mission, the rover might have to wait for a communication opportunity or spend time transmitting large datasets. Adding this would require durative offloading actions or additional PDDL+ processes for transmission.

Corruption is deterministic and linear. Each dataset has a fixed corruption rate, and corruption increases continuously at that rate while the data is stored. Real memory corruption may be stochastic, for example due to radiation effects or hardware faults. The deterministic abstraction makes the model easier to test and the results easier to interpret, but it cannot express probabilities, risk levels or random failures.

Movement is instantaneous in the PDDL+ version. Therefore, corruption and encoding evolve during explicit waiting periods, but not during realistic travel time. This keeps the model compatible with the selected planner and keeps the focus on data management, but it reduces physical realism. A future version could introduce durative movement or PDDL+ movement processes so that long routes also affect corruption and encoding.

The model also assumes a single rover, perfect knowledge and deterministic actions. The rover knows the full map, all dataset locations, all data sizes, and all corruption and encoding parameters. Actions always succeed when their preconditions are satisfied. The model does not represent sensing uncertainty, terrain difficulty, obstacle avoidance, battery constraints, hardware failures, or cooperation between multiple rovers.

These limitations are acceptable for this assignment because the objective is to study memory constraints and data management, not to build a complete rover autonomy stack. Adding all physical details would make the model much more complex and could obscure the main planning issue. The current abstraction keeps the problem focused, testable and interpretable.

7.3 Known Issues and Possible Extensions

The main known issue is the simplified treatment of time and movement. Since movement is instantaneous, the distance from the base affects the number of actions but not the continuous mission time. This means that timing feasibility depends mainly on encoding and corruption parameters, not on route duration. A natural extension would be to assign travel durations to movements.

Another possible extension would be to model offloading as a durative or continuous process. Large datasets could require more time to transmit, and communication could be possible only at specific locations or time windows. This would create a stronger interaction between memory usage, timing and communication planning.

A further extension would be to add optional data deletion or compression actions. For example, the rover could choose to discard low-priority data to make space for higher-priority observations, or compress data at the cost of additional processing time. These features were not included because the assignment focuses on required data collection and safe offloading.

Overall, the model deliberately balances realism and clarity. It is abstract enough to remain understandable, but expressive enough to demonstrate numeric memory constraints, autonomous corruption, encoding delays, automatic data loss and timing-dependent feasibility.

8 Conclusion

The project implements the requested memory-constrained rover planning model in both classical PDDL and PDDL+. The classical part uses numeric fluents to represent memory capacity and shows that limited storage changes the generated plan. The PDDL+ part adds continuous encoding and corruption, together with automatic events for encoding completion and data loss. The solvable and unsolvable instances show that both memory capacity and timing affect mission feasibility. The model is intentionally abstract, but it captures the core planning trade-offs required by the assignment and remains interpretable, testable and reproducible.

9 Reproducibility

The planner was executed in WSL using Java and ENHSP. The following examples show the exact style of command used; raw outputs were saved in `plans/`.

```
java -jar ~/enhsp/ENHSP-Public/enhsp-dist/enhsp.jar \  
-o /mnt/c/Users/Endri/Desktop/D1-V8-memory-rover/pddl/domain-memory-rover.pddl \  
-f /mnt/c/Users/Endri/Desktop/D1-V8-memory-rover/pddl/problem-2-memory-constrained.pddl \  
-planner opt-hrmax
```

```
java -jar ~/enhsp/ENHSP-Public/enhsp-dist/enhsp.jar \  
-o /mnt/c/Users/Endri/Desktop/D1-V8-memory-rover/pddl-plus/domain-memory-rover-plus.pddl \  
-f /mnt/c/Users/Endri/Desktop/D1-V8-memory-rover/pddl-plus/problem-plus-4-multiple-failures.pddl
```